Online Shopping Comparison DBMS

Project Team:

Mallhar Arvind Totey 250041026	Aditya Barman 250001005	Aryan Gothwal 250001010

1. Introduction:

1.1 Background of the project:

In recent times e-commerce has seen a rapid growth due to more and more people getting chronically online. E-commerce has given people the ability to buy a variety of things at just a click of their fingers. It is convenient to use, provides the user with tons of products sold from all sorts of places. Another thing is that many such e-commerce platforms exist and oftentimes all of them sell the same product at different prices. Obviously a user would want to get the product at the best price possible. The thing which is not so convenient though, as a user, is to keep track of all the platforms where the product is available, study its price history, read its reviews at each of those platforms, see about coupons, offers, delivery charges, time required for delivery and after considering all such factors that matter the user picks the option that is most optimal for them. As is clear, this process can be very time consuming, repetitive and inconvenient for the user at times.

2. Online shopping comparison DBMS:

2.1 Our Project: The proposed solution

2.1.1 The Problem Definition:

The problem that is tacked by the project:
"The lack of a convenient system for comparing product prices across multiple e-commerce platforms and evaluating whether the current price is actually a good deal or not ."

2.1.2 Proposed Solution:

The proposed solution is a web-based, database-driven application. This application will help the user with the exact problem stated above. the application compares **products, prices, offers and historical price data**.

Brief Functioning of the application:

- The product and offer information will be obtained through available APIs provided by the e-commerce platforms themselves or from the other permitted sources.
- The collected data and information will be processed and stored in the database.
- Also instead of relying on the latest price, the system will maintain historical records of prices and offers, this can be used to give the user useful insights.

- Furthermore, in the application the user will be able to do product search, then the system will display them with a list of options and also show them the price history.

- The users will also be able to maintain a shopping cart to store some items that they would like to buy at a later time. The items in the shopping cart will be tracked for providing alerts.

- The users will also be able to see a summary of the reviews platform wise.

---

3. Database Details:

3.1 Importance of Database:

The application will have to manage a large number of related entities, including users, products, stores, product listings, offers, price records, alerts, watchlists and categories.

The database will therefore not just store the current prices, it will also store some historical data and information about the product and the relationships between multiple other entities of the DB.

3.2 Database Entities and the conceptual schema of the DB:

a. **The User entity** keeps track of registered users. It contains user ID, name, email, password hash, account status and registration date. The user ID is the PK and has unique and NOT NULL constraints. Email must be unique and NOT NULL. Name and password hash are NOT NULL. Account status can have values such as ACTIVE or BLOCKED and has a default value of ACTIVE. Registration date is NOT NULL. None of these attributes are multivalued.

b. **The Product Entity:** Each product is described by a set of attributes product ID, product name, brand, description, category and product specifications. The product ID is the primary key and has a unique and NOT NULL constraint. Product name and brand are NOT NULL. Description may be NULL. Category is represented by a foreign key referring to Category.

c. **Category Entity:** Each category is described by a set of attributes category ID, category name and description. Category ID is the primary key and is unique and NOT NULL. Category name is unique and NOT NULL, while description may be NULL. None of the

attributes are multivalued.

d. **Store Entity:** Each store is described by the following set of attributes store ID, store name, website URL, and store status. Store ID is the primary key and has a unique and NOT NULL constraint. Store name and website URL are unique and NOT NULL. Store status may contain values such as ACTIVE or INACTIVE. None of the attributes are multivalued.

e. **Store Listing Entity**: A Store Listing is meant to represent a particular product listed on a particular store. It contains listing ID, product ID, store ID, store-specific product ID, current price, availability, seller name, product URL and last updated time. Listing ID is the primary key and is unique and NOT NULL. Product ID and Store ID are foreign keys and are NOT NULL. Store-specific product ID is NOT NULL and should be unique together with Store ID. Current price must be NOT NULL and greater than or equal to zero. Availability may have values such as IN_STOCK and OUT_OF_STOCK. Seller name may be NULL. Product URL and last updated time are NOT NULL. None of the attributes are multivalued.

f. **Price History Entity:** Price History will store the historical prices of a particular store listing of that product. It contains price record ID, listing ID, price and recorded timestamp. Price record ID is the primary key and is unique and NOT NULL. Listing ID is a NOT NULL foreign key referring to Store Listing. Price must be NOT NULL and greater than or equal to zero. The recorded timestamp is NOT NULL. None of the attributes are multivalued.

g. **Price Alert Entity:** Price Alert will store the conditions specified by users for receiving notifications. This will be necessary in tracking the cart of the user. It contains alert ID, user ID, product ID, target price, alert status, created date and triggered date. Alert ID is the primary key and is unique and NOT NULL. User ID and Product ID are NOT NULL foreign keys. Target price must be greater than zero. Alert status may contain ACTIVE, TRIGGERED or DISABLED. Created date is NOT NULL, while triggered date may initially be NULL. None of the attributes are multivalued.

h. **Watchlist Entity:** Watchlist has the products that a particular user wants to track in his cart. It contains user ID, product ID and added date. User ID and Product ID together form the composite primary key and are also foreign keys referring to User and Product. The added date is NOT NULL. None of the attributes are multivalued.

i. **Offer Entity:** Offer will contain discounts, coupons, cashback or other promotional offers associated with the product and store. It will contain offer ID, listing ID, offer type, offer value, coupon code, description, valid-from date, valid-until date and offer status. Offer ID is the primary key and is unique and NOT NULL. Listing ID is a NOT NULL foreign key referring to Store Listing. Offer type can contain values such as COUPON, DISCOUNT or CASHBACK. Offer value must be greater than or equal to zero. Coupon code and description may be NULL. Valid-from is NOT NULL and valid-until must be later than valid-from. Offer status may contain ACTIVE or EXPIRED.

4. Provisional Architecture:

We are planning to make the system use APIs or other permitted data sources provided by the various e-commerce platforms.

E-commerce APIS → Data Processing → Data Ingestion into the DB → Backend REST API → Frontend → User

The database will act as the central persistent layer rather than requiring the frontend to directly retrieve information from every external marketplace.

Repeated collection of product and price data will allow the system to gradually construct its own historical dataset in a way.

5. Conclusion:

The proposed **Price Comparison and Deal Intelligence System** application is aimed at providing the users with a centralized platform for comparing products, prices, offers, and price histories across multiple e-commerce stores. By storing this information in a structured relational database, the system can provide useful features such as price comparison, historical price analysis, watchlists, and price alerts.

Along with the Database details mentioned above, the other things that we have to do to complete our product are:
creation of a frontend, backend API layer, external api integration.

Overall, the project combines a **practical real-world problem with meaningful database functionality**.